COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE 21S1
DATE PERFORMED:	07/23/2026	**DATE SUBMITTED:**	07/23/2026
NAME:	Kenji Baltazar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5 **Hands-on Activity 3.1 Linked Lists**

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

Screenshot

CODE

```
#include<iostream>
#include<utility>
class Node{
    public:
        char data;
        Node *next;
    };
int main(){
    //step 1
    Node *head = NULL;
    Node *second = NULL;
    Node *third = NULL;
    Node *fourth = NULL;
    Node *fifth = NULL;
    Node *last = NULL;
    //step 2
    head = new Node;
    second = new Node;
    third = new Node;
```

```
19      third = new Node;
20      fourth = new Node;
21      fifth = new Node;
22      last = new Node;
23      //step 3
24      head->data = 'C';
25      head->next = second;
26      second->data = 'P';
27      second->next = third;
28      third->data = 'E';
29      third->next = fourth;
30      fourth->data = '0';
31      fourth->next = fifth;
32      fifth->data = '1';
33      fifth->next = last;
34      //step 4
35      last->data = '0';
36      last->next = nullptr;
37  }
```

OUTPUT

```
PS C:\Users\ACER NITRO\OneDrive\Documents\C++ Project\LINKED LIST>
```

NOTHING WAS PRINTED

Discussion	The code was alright, it was working, no errors and bugs but it lacks one thing, a traversal step to print out the intended output.

Table 3-1. Output of Initial/Simple Implementation

Operation	**Screenshot**

Traversal

```
template <typename T>
void listTraversal(SingleList<T>* head){

    while(head != nullptr){
        std::cout<<head->data;

        //to check if the next line is not equal to null
        if(head->next != nullptr){
            std::cout<<"->";
        }
        head = head->next;
    }
    std::cout<<std::endl;
}
```

Insertion at head

```
template <typename T>
void sllinsertHead(T newData, SingleList<T> ** currentHead){

    //Allocate memory for the new node
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    //Set next of the new node to point to the previous Head
    newNode->next = *currentHead;
    //reset head to point to the new node
    *currentHead = newNode;
}
```

Insertion at any part of the list

```
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T> * prevNode){
    //check if it is the head node (previous node is null)
    if(prevNode==nullptr){
        std::cout<<"The previous node cannot be null."<<std::endl;
        return;
    }

    SingleList<T>* newNode = new SingleList<T>;
    //store data in the new node
    newNode->data = newData;
    //point new node to the node prev node was pointing to
    newNode->next = prevNode->next;
    //point prev node to the new node
    prevNode->next = newNode;
}
```

Insertion at the end

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;
    //dereference to the head node
    SingleList<T>* currentNode = *head;
    //store data in new data
    newNode->data = newData;
    //point next of new node to null
    newNode->next = nullptr;
    //check if the list is empty
    if(currentNode == nullptr){
        currentNode == newNode;
        return;
    }
    //traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        //iteration going to one node to another
        currentNode = currentNode->next;
    }
    currentNode->next = newNode;
}
```

Deletion of a node

```
template <typename T>
void sllDelete(T findData, SingleList<T>** head){
    //check if there is a list
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

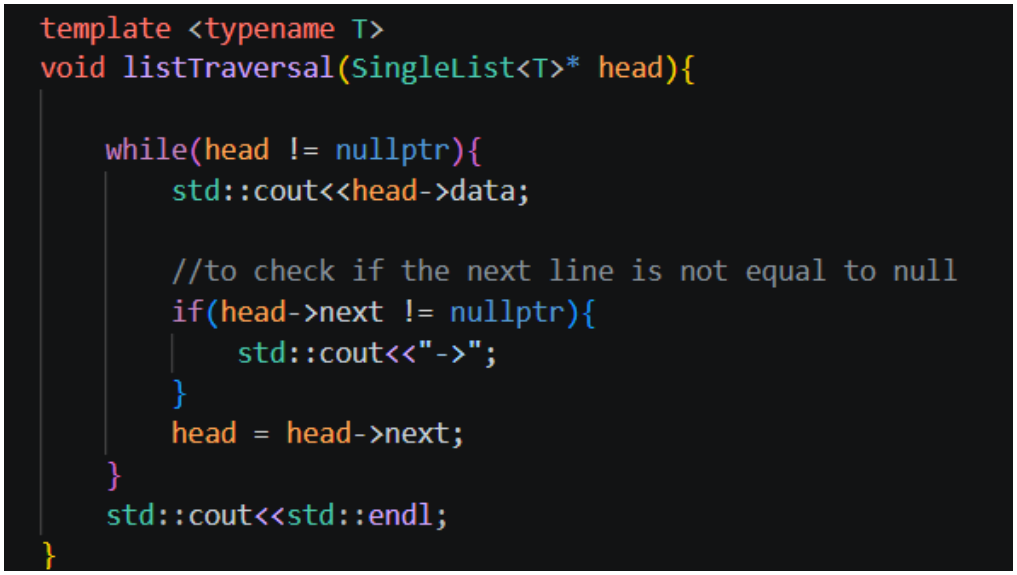
    //Search for the node:
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    //if data is not found
    if (currNode == nullptr) return;
```

```
    //delete the head node(if the head ndode is the findData)
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }

    delete currNode;
}
```

Table 3-2. Code for the List Operations

a	CODE	
	ANALYSIS	This is the header file name, with a **SingleList** class having a public variable for **T** data and a pointer to the next node in the list.
b	CODE	
	ANALYSIS	This function walks through the linked list from the head node to the end, printing each node's data separated by arrows, until it reaches the last node.

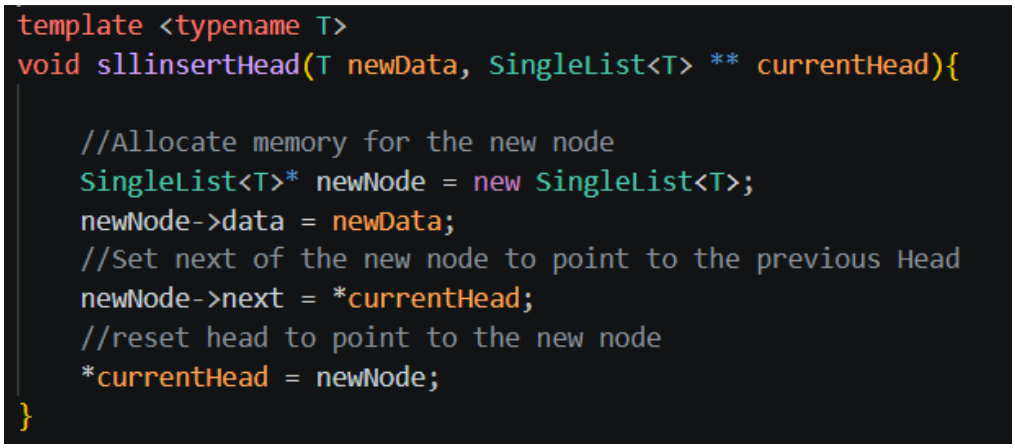
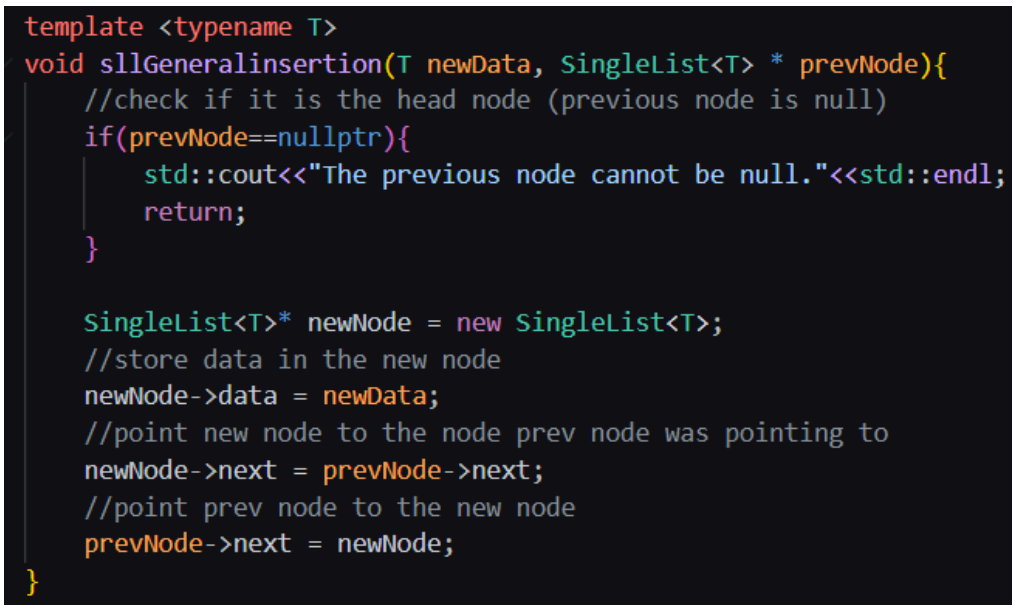
```
#ifndef SINGLY_LL_H //header file name
#define SINGLY_LL_H //

template <typename T>
class SingleList{
    public:
    T data;
    SingleList<T> *next = nullptr;
};
```

```
template <typename T>
void listTraversal(SingleList<T>* head){

    while(head != nullptr){
        std::cout<<head->data;

        //to check if the next line is not equal to null
        if(head->next != nullptr){
            std::cout<<"->";
        }
        head = head->next;
    }
    std::cout<<std::endl;
}
```

c	CODE	
	ANALYSIS	This function inserts a new node at the beginning of the list then it creates a new node, points its next to the old head, then updates the head pointer to point to the new node.
d	CODE	
	ANALYSIS	This function inserts a new node right after the **prevNode**, it first makes sure that the **prevNode** is not a null, then links the new node between **prevNode** and whatever is the next.

```
template <typename T>
void sllinsertHead(T newData, SingleList<T> ** currentHead){

    //Allocate memory for the new node
    SingleList<T>* newNode = new SingleList<T>;
    newNode->data = newData;
    //Set next of the new node to point to the previous Head
    newNode->next = *currentHead;
    //reset head to point to the new node
    *currentHead = newNode;
}
```

```
template <typename T>
void sllGeneralinsertion(T newData, SingleList<T> * prevNode){
    //check if it is the head node (previous node is null)
    if(prevNode==nullptr){
        std::cout<<"The previous node cannot be null."<<std::endl;
        return;
    }

    SingleList<T>* newNode = new SingleList<T>;
    //store data in the new node
    newNode->data = newData;
    //point new node to the node prev node was pointing to
    newNode->next = prevNode->next;
    //point prev node to the new node
    prevNode->next = newNode;
}
```

e	CODE	

```
template <typename T>
void sllinsertEnd(T newData, SingleList<T>** head){
    //Allocate new node
    SingleList<T>* newNode = new SingleList<T>;
    //dereference to the head node
    SingleList<T>* currentNode = *head;
    //store data in new data
    newNode->data = newData;
    //point next of new node to null
    newNode->next = nullptr;
    //check if the list is empty
    if(currentNode == nullptr){
        currentNode == newNode;
        return;
    }
    //traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        //iteration going to one node to another
        currentNode = currentNode->next;
    }
    currentNode->next = newNode;
}
```

	ANALYSIS	This function is for inserting a new node at the end of the list. It goes up to the last node and then links it to a new node.
f	CODE	

```
template <typename T>
void sllDelete(T findData, SingleList<T>** head){
    //check if there is a list
    if(*head == nullptr) return;

    SingleList<T>* currNode = *head;
    SingleList<T>* prevNode = nullptr;

    //Search for the node:
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    //if data is not found
    if (currNode == nullptr) return;
```

		``
		    //delete the head node(if the head ndode is the findData)
		    if(prevNode == nullptr){
		        *head = currNode->next;
		    }else{
		        prevNode->next = currNode->next;
		    }
		
		    delete currNode;
		}
		``

	ANALYSIS	This function deletes a node that has **findData**. It searches through the linked list then, if found, it links the previous node to the found node's next. Then it frees the memory of the removed node with the **delete currNode**.

Table 3-3. Code and Analysis for Singly Linked Lists

CODE

```
#ifndef DOUBLE_LL_H
#define DOUBLE_LL_H

#include <iostream>

template <typename T>
class doubleList{
    public:
        T data;
        doubleList<T>* next = nullptr;
        doubleList<T>* prev = nullptr;
};
```

```cpp
//====================
//creating a node
//====================
template <typename T>
doubleList<T>* new_node(T newData){
    //dynamically allocate memory for a new node
    doubleList<T>* newNode = new doubleList<T>;
    //store data in the node:
    newNode->data = newData;
    //since its a new node, it will point to null
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}
```

CODE

```
//====================
//traversal
//====================
template <typename T>
void dllTraverse(doubleList<T>* currentNode){
    doubleList<T>* tail = nullptr;
    while(currentNode != nullptr){
        //prints the data
        std::cout << currentNode->data << " ";
        //save the current node as the last visited node
        tail = currentNode;
        //move to the next node
        currentNode = currentNode->next;
    }
    std::cout << std::endl;

    //reverse traversal
    std::cout << "Reverse Traversal: ";

    while(tail != nullptr){
        std::cout << tail->data << " ";
        tail = tail->prev;
    }

    std::cout << std::endl;
}
```

OUTPUT

```
initial Traversal
E N J
Reverse Traversal: J N E
```

```
Reverse Traversal: J N E K
Reverse Traversal: I J N E K
Reverse Traversal: I J X N E K
Reverse Traversal: J X N E K
```

CODE

```
//====================
//insertion at the head
//====================
template <typename T>
void dllInsertHead(T newData, doubleList<T>** currentHead){
    //create a new node
    doubleList<T>* newNode = new_node(newData);

    //SET THE new node to point to the current head
    newNode->next = *currentHead;

    //current head will be pointed to the new node
    if(*currentHead != nullptr){
        (*currentHead)->prev = newNode;
    }
    //update the head pointer
    *currentHead = newNode;
}
```

OUTPUT

```
inserting at the head
K E N J
```

CODE

```
//====================
//general insertion
//====================
template <typename T>
void dllInsertAny(T newData, doubleList<T>* prevNode){
    //cannot insert after a null (prevNode cannot be null -> return)
    if (prevNode == nullptr) {
        std::cout << "The previous node cannot be null." << std::endl;
        return;
    }

    //create a new node
    doubleList<T>* newNode = new_node(newData);
```

```
    //new node points to the new node after the prevNode
    newNode->next = prevNode->next;

    //newNode previous mustbe pointed to the prevNode
    newNode->prev = prevNode;

    //if there is a node after the prevNode, make it point back to the new node
    if (prevNode->next != nullptr) {
        prevNode->next->prev = newNode;
    }
    //prevNode will be pointed to the new node
    prevNode->next = newNode;
}
```

OUTPUT

```
inserting at any node
K E N X J I
```

CODE

```cpp
//====================
//insertion at the end
//====================
template <typename T>
void dllInsertEnd(T newData, doubleList<T>* currentHead){
    //create a new node
    doubleList<T>* newNode = new_node(newData);
    //traverse until the last node
    doubleList<T>* currentNode = currentHead;
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    //connect the last node to the new node
    currentNode->next = newNode;

    //connect the prev node to the new node
    newNode->prev = currentNode;
}
```

OUTPUT

```
inserting at the end
K E N J I
```

CODE

```
//====================
//deletion
//====================
template <typename T>
void dllDelete(T findData, doubleList<T>** currentHead){
    //traverse until the node that we want to delete is found
    doubleList<T>* currentNode = *currentHead;
    while(currentNode != nullptr && currentNode->data != findData){
        currentNode = currentNode->next;
    }

    if(currentNode == nullptr){ //so when we reach the end of the list which is null, we will return
        std::cout << "Node not found" << std::endl;
        return;
    }
```

```
    if(currentNode == *currentHead){ //if the node to be deleted is the head node
        *currentHead = currentNode->next; //update the head pointer to the next node
        if(*currentHead != nullptr){
            (*currentHead)->prev = nullptr; //set the prev pointer of the new head to null
        }
    } else {
        /*this one if the node to be deleted is not the head node, we connect the previous node
        to th next node and the next node to the previous node*/
        currentNode->prev->next = currentNode->next;
        if(currentNode->next != nullptr){
            currentNode->next->prev = currentNode->prev;
        }
    }

    //delete the node
    delete currentNode;
}
```

OUTPUT

```
Delete node
K E N X J
```

```
#endif
```

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

playlist_LL.h

```h
#ifndef PLAYLIST_LL_H
#define PLAYLIST_LL_H

#include <iostream>

template <typename T>
class kantaNode{
    public:
        T data;
        kantaNode<T>* next = nullptr;
        kantaNode<T>* prev = nullptr;
};
```

```h
//====================
//creating a node
//====================
template <typename T>
kantaNode<T>* new_song(T newData){
    //dynamically allocate memory for a new node
    kantaNode<T>* newNode = new kantaNode<T>;
    //store the data in the node:
    newNode->data = newData;
    //since its a new node, it will point to null for now
    newNode->next = nullptr;
    newNode->prev = nullptr;

    return newNode;
}
```

Analysis: This is for creating a new node and puts the data inside that node. Each node is for one song, and has a pointer to the next song, and a pointer to the previous song. This means that having next and prev

means that we can move to the next and the prev song through the playlist.

```
//====================
//Traversal
//====================
template <typename T>
void playlisttraverse(kantaNode<T>* currentHead){
    if(currentHead == nullptr){
        std::cout << "Playlist is empty.\n";
        return;
    }
    kantaNode<T>* currentNode = currentHead;
    do{
        std::cout << currentNode->data << std::endl;
        currentNode = currentNode->next;
    } while(currentNode != currentHead);
}
```

Analysis: This is to print out every song that I have in the playlist. I added if statement if the currentHead is in null and returns the statement “Playlist is empty”. I used a do-while loop which starts at the head, prints each song in the playlist, then moves to the next one, and it keeps on going until it is back on the head again.

```
//====================
//move to the next song
//====================
template <typename T>
void playlistNext(kantaNode<T>** current){
    if(*current == nullptr) return;
    *current = (*current)->next;
    std::cout << "Now playing: " << (*current)->data << std::endl;
}
```

```cpp
//====================
//move to the previous song
//====================
template <typename T>
void playlistPrev(kantaNode<T>** current){
    if(*current == nullptr) return;
    *current = (*current)->prev;
    std::cout << "Now playing: " << (*current)->data << std::endl;
}
```

Analysis: These 2 function moves the current song forward or backward.

```cpp
//====================
//insertion at the end
//====================
template <typename T>
void playlistInsert(T newData, kantaNode<T>** currentHead){
    //create a new node
    kantaNode<T>* newNode = new_song(newData);
    //if the playlist is empty, the new node points to itself
    if(*currentHead == nullptr){
        newNode->next = newNode;
        newNode->prev = newNode;
        *currentHead = newNode;
        return;
    }
```

```
    //the last node is the one before the head
    kantaNode<T>* last = (*currentHead)->prev;
    //connect the last node to the new node
    last->next = newNode;
    newNode->prev = last;
    //connect new node back to the head
    newNode->next = *currentHead;
    (*currentHead)->prev = newNode;
}
```

Analysis: This adds a new song at the bottom of the playlist. Then If the playlist is empty, the new song becomes the only song, and it points to itself. If not empty, the function finds the last song using **(*currentHead)->prev**, then it links the last song to the new song, and links the new one back to head node.

```
//====================
//deletion
//====================
template <typename T>
void playlistDelete(T findData, kantaNode<T>** currentHead){
    if(*currentHead == nullptr){
        std::cout << "Playlist is empty." << std::endl;
        return;
    }
    //traverse the circle until node is found or we go back to head
    kantaNode<T>* currentNode = *currentHead;
    do{
        if(currentNode->data == findData) break;
        currentNode = currentNode->next;
    } while(currentNode != *currentHead);
```

```
if(currentNode->data != findData){ //if didn't find it
    std::cout << "Song not found." << std::endl;
    return;
}
if(currentNode->next == currentNode){ //only song left in the playlist
    *currentHead = nullptr;
} else {
    //connect the previous node to the next node and vice versa
    currentNode->prev->next = currentNode->next;
    currentNode->next->prev = currentNode->prev;

    if(*currentHead == currentNode){ //if we deleted the head, move the currenthead to the next
        *currentHead = currentNode->next;
    }
}
```

Analysis: This function is for deletion. If the pointer points to null, the playlist prints out “Playlist is empty”. This searches for a song that has findData by traversing in the playlist using do-while loop. Then after traversing and if it doesn’t find the song, it prints out “Song not found”. Then another if statement if there is only one song left, it sets the currenthead as a nullptr to avoid pointing to a deleted memory since it is pointing to itself. Then else, if there are more than one song, it connects the prev node to the next node and does it in reverse as well, connecting next node to the prev node. Then If the deleted song is the head, the head pointer is moved to the next.

```
    //delete the node
    delete currentNode;
}

#endif
```

Analysis: To delete the node or free the memory

playmain.cpp

```cpp
#include <iostream>
#include "playlist_LL.h"

int main(){
    kantaNode<std::string>* head = nullptr;
    kantaNode<std::string>* current = nullptr;

    playlistInsert<std::string>("Bahay Kubo", &head);
    playlistInsert<std::string>("Girl in the Mirror", &head);
    playlistInsert<std::string>("Shinunoga E-wa", &head);
    current = head;

    std::cout << "Play all:" << std::endl;
    playlisttraverse(head);

    std::cout << std::endl << "Next:" << std::endl;
    playlistNext<std::string>(&current);
    std::cout << std::endl << "Previous:" << std::endl;
    playlistPrev<std::string>(&current);

    std::cout << std::endl << "Remove Girl in the Mirror:" << std::endl;
    playlistDelete<std::string>("Girl in the Mirror", &head);
    playlisttraverse(head);

    return 0;
}
```

Output

```
Play all:
Bahay Kubo
Girl in the Mirror
Shinunoga E-wa

Next:
Now playing: Girl in the Mirror

Previous:
Now playing: Bahay Kubo

Remove Girl in the Mirror:
Bahay Kubo
Shinunoga E-wa
```

In this activity, I learned how linked lists work — how each piece (node) connects to the next using pointers, and how adding or removing items just means changing these connections instead of moving the actual data. I also learned why we sometimes need a "pointer to a pointer" (**) — it's so a function can actually change the original head variable, not just a copy of it. The steps we followed built up nicely: first the basic list and how to move through it, then how to add items, then how to remove them. The extra activity (the playlist example) took this further by using a list that connects both forward and backward (next and prev), and loops back to the start instead of ending at nullptr — like a real playlist that repeats. It also showed why we need to handle special cases, like when only one song is left, so the program doesn't end up pointing to memory that's already been deleted. Overall, I think I understood the material well and could follow the logic once I broke it down piece by piece, but I still want to practice writing these functions myself from scratch, tracing pointer changes step-by-step (maybe by drawing them out), and spotting tricky cases on my own instead of only noticing them after they're explained.

Conclusion: Overall, in this activity, I learned a great deal of information regarding singly and double linked list. I also learned how each nodes connects to the the nodes using pointers and how adding and removing these nodes is much more dynamic than arrays as each nodes is adjustable by just adding a pointer. The first we did was to make a singly list and how to move through it, then to add nodes then to also remove them, it is similar as well to the double linked list but the difference is that it can traverse back to the head. The supplementary also took this to the next level because it now uses a real life representation like the playlist. I was able to do this by using a linked list that connect forward and backward and loops again to the start instead of ending in nullptr. Overall, I somehow was able to adjust at the linked list concept as well as the break down of it which is just add a node, traverse, insert front, general, end, then delete. It was a hard task to do and took me a while to really digest all this but somehow I am getting there.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Error checkings and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **AI was used to help me identify and troubleshoot my errors then it assisted me to check when I missed a code or something. AI did not replace my original idea as it was mainly based off in the procedural code and code from our professor.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.